SIMATS ENGINEERING

ITA14 – ETHICAL HACKING

ASSIGNMENT 3 – CO3

Engineering a Reusable Scan-Data Processing and Reporting Pipeline

Mode: Pair work – 2 students per group

Particulars	**Details**
Student A Student B	Balaji Rohith MV | Register No.: 192521269 Chokkalingam S | Register No.: 192524111
Department / Section	Information Technology
Academic Year	2025–2026
Duration / Total Marks	1 Week / 50 Marks per student
Assignment Mode	Pair Work – both students participate in every question

Introduction and Scope

This assignment develops a repeatable pipeline for importing, validating, cleaning, reshaping, merging, exporting and reporting authorized network-scan data. The workflow uses the tools and concepts specified in the assignment brief, including Nmap, enum4linux, nbtscan, ADB, shell utilities, CSV datasets, joins, melting and casting, Bash functions, checksums and performance measurements.

All scanning must be limited to systems owned by the students or explicitly covered by written authorization. Use an isolated lab subnet and synthetic datasets for defect injection and performance tests. Do not scan public or third-party systems without

explicit permission. Commands involving OS detection, UDP, vulnerability scripts, ADB, RMI, NCP or Bluetooth enumeration must only be run where the relevant target and technique are in scope.

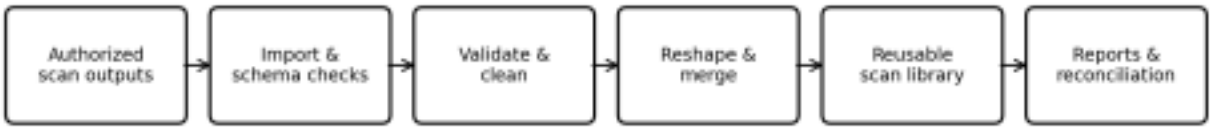


Figure 1. Proposed scan-data processing pipeline.

Pair-work and evidence principles

Both students participate in every question. Student A and Student B have different primary responsibilities, but neither student should work only on their assigned portions. Each student should retain their initial proposal, scripts, test cases, command outputs and explanation. The contribution record near the end must be completed honestly with the work actually performed.

The examples below are reproducible templates. They do not claim that scans or benchmarks have already been executed. Replace sample data with outputs from the pair's authorized lab, and record actual counts, timings and checksums rather than inventing results.

Q1 – Investigate Requirements and Create the Scan-Data Architecture

1.1 Requirements and datasets

The pipeline needs a consistent representation of hosts, ports, services,

operating-system/version observations and enumeration findings. Each dataset should have a stable key, a source reference and collection metadata so that records can be traced to a particular scan session.

Dataset	Suggested CSV columns Key / relationship
Hosts	scan_id, ip, hostname, status, mac, vendor, os_guess, discovered_at, source_file — Primary key: scan_id + ip; one host can have many ports.
Ports	scan_id, ip, protocol, port, state, reason, discovered_at — Key: scan_id + ip + protocol + port; links to Hosts.
Services	scan_id, ip, protocol, port, service_name, product, version, extra_info — Links to Ports using scan_id + ip + protocol + port.
OS_Versions	scan_id, ip, os_name, os_family, os_gen, accuracy, method — Links to Hosts using scan_id + ip.
Enumeration	scan_id, ip, port, category, finding, evidence, tool, collected_at — relationship where relevant.
	Links to host; optional port

1.2 Relationship model

Parent dataset	Child dataset Relationship rule
Hosts	Ports One host may have zero or many port records.
Ports	Services — A port may have zero or one or multiple service observations across scan sessions.

Hosts	OS_Versions One host may have multiple OS observations over time.
Hosts / Ports	Enumeration Enumeration can be host-level or associated with a specific port.
All datasets Scan metadata	scan_id identifies the run, scope, time, tool and command options.

1.3 Relationship diagram

Entity	Primary key Foreign key / link
Hosts	scan_id, ip scan_id references ScanSessions
Ports	scan_id, ip, protocol, port scan_id + ip references Hosts
Services	scan_id, ip, protocol, port, observation_id port key references Ports
OS_Versions	scan_id, ip, observation_id scan_id + ip references Hosts
Enumeration	scan_id, ip, finding_id scan_id + ip references Hosts; optional port key

1.4 CSV header templates

```
hosts.csv: scan_id,ip,hostname,status,mac,vendor,os_guess,discovered_at,source_file
ports.csv: scan_id,ip,protocol,port,state,reason,discovered_at
services.csv: scan_id,ip,protocol,port,service_name,product,version,extra_info
os_versions.csv: scan_id,ip,os_name,os_family,os_gen,accuracy,method
enumeration.csv: scan_id,ip,port,category,finding,evidence,tool,collected_at
```

1.5 Testable enquiry questions and acceptance criteria

Enquiry question	Example Kali/shell query Acceptance criteria
Which hosts were discovered in a scan session?	awk -F, 'NR>1 {print $2}' hosts.csv | sort -u Every returned IP is valid, unique and belongs to that scan_id.

Which TCP ports are open on each host?	awk -F, 'NR>1 && $3=="tcp" && $5=="open" {print $2 "," $4}' ports.csv	Only valid ports 1–65535 are included; no duplicate host/protocol/port key.
Which services have version information?	awk -F, 'NR>1 && $6!="" {print $2,$4,$6,$7}' services.csv	Each service maps to a matching port record or is flagged unmatched.

Which hosts have OS or enumeration observations?	sort -u; cut -d, -f2 enumeration.csv | sort -u	Hosts or are recorded as exceptions.
cut -d, -f2 os_versions.csv |	All host references exist in	

Student A leads the Hosts and Ports schemas and requirements-to-column mapping. Student B leads the service, OS/version and enumeration relationships and the enquiry questions. Both should review the complete model and explain the keys.

Q2 – Create and Validate the Source Scan Datasets

2.1 Authorized data collection

Use a dedicated lab subnet such as 192.168.56.0/24 only when it is configured and owned by the pair. Save original outputs separately from transformed CSV files. Prefer Nmap XML as a structured source and preserve the original XML for audit.

```
nmap -sn 192.168.56.0/24 -oX hosts_discovery.xml
nmap -sT -p 1-1024 192.168.56.10 -oX tcp_ports.xml
nmap -sU --top-ports 20 192.168.56.10 -oX udp_ports.xml
nmap -sV -p 22,80,443 192.168.56.10 -oX service_versions.xml
# OS detection only on explicitly authorized lab targets
nmap -O 192.168.56.10 -oX os_detection.xml
```

For other enumeration tools named in the brief (enum4linux, nbtscan, adb, rmi-dumpregistry, ncp-serverinfo and sdptool), use them only against a lab device configured for that protocol and only when the relevant access is authorized. Record tool

version, command, target, timestamp and output file.

2.2 Validation and standardization rules

Validation	Rule Failure handling
Required columns	CSV header must match the dataset schema exactly. Reject file or route to schema-review queue.
IP address	Must parse as IPv4/IPv6; do not rely only on a loose regex. Reject malformed address; retain original row in rejected.csv.
Port	Integer in range 1–65535; protocol must be tcp or udp. Reject invalid values; port 0 and 65536 are boundary failures.
Service name	Normalize through a controlled lookup file. Unknown variant is flagged for review, not guessed.
Duplicates	Unique key per scan_id, IP, protocol and port. Deduplicate only after preserving all source provenance.
Scope	Target must match the approved asset list. Reject out-of-scope rows and record the reason.

2.3 Controlled defects and test data

```
cp ports.csv ports_testcopy.csv
# Introduce defects only into the test copy, never the raw scan evidence. #
Example: duplicate a row and alter a service label for validation exercises.
tail -n 1 ports_testcopy.csv >> ports_testcopy.csv
cp services.csv services_testcopy.csv
sed -i '2s/http/HTTP/' services_testcopy.csv
```

Defect case	Injected condition Expected pipeline response
Missing value	Required port or IP field is blank. Reject or route to manual review; never silently fill a

	value.
Duplicate key	Same scan_id, IP, protocol and port appears twice. Keep one normalized record and log the duplicate.
Inconsistent service	http, HTTP and www appear as variants. Map known aliases; unknown labels go to review.
Out-of-scope address	A row contains an IP not in the approved inventory. Reject before merging.
Invalid port	Port is 0 or 65536. Reject as outside valid port range.

2.4 Large-pipeline validation order

- Check file existence and schema/header first, so malformed files do not enter processing.
- Apply scope validation immediately after parsing target identifiers; out-of-scope records must not proceed.
- Validate IP, protocol and port types/ranges.
- Normalize known service aliases and preserve unknown values for review.
- Deduplicate using the complete composite key while retaining provenance.
- Check cross-dataset relationships and report unmatched records.

Student A constructs Hosts and Ports CSVs and controlled defects. Student B constructs Services, OS/Versions and Enumeration CSVs and implements validation and standardization. The group should compare predicted pass/fail/review counts with actual outputs.

Q3 – Build a Scan-Data Editing and Cleaning Module

3.1 Reusable cleaning operations

Cleaning rules should be deterministic and conservative. Automatically correct only cases where the intended value is unambiguous. Missing IP addresses, unknown service labels and conflicting observations should be sent to review rather than guessed.

```
clean_ports() {
 local input="$1" output="$2" rejected="$3"
 awk -F, 'BEGIN{OFS=","} NR==1{print;next} \
 $2=="" || $3=="" || $4=="" {print > "rejected.csv"; next} \
 $4 !~ /^[0-9]+$/ || $4<1 || $4>65535 {print > "rejected.csv"; next} \
{print}' "$input" > "$output"
}
# Deduplicate after sorting by the composite key; preserve header separately. (head
-n 1 ports.csv; tail -n +2 ports.csv | sort -t, -k1,1 -k2,2 -k3,3 -k4,4 -u) >
ports_dedup.csv
```

The illustrative function shows the logic, but a production version should pass the rejected-file path into awk with -v, quote all paths, clear the rejected file at the start and avoid hard-coded filenames. For CSV fields containing quoted commas, use a CSV-aware parser rather than simple awk field splitting.

3.2 Service-name lookup

```
alias,canonical
www,http
HTTP,http
http-alt,http-alt
ssh,ssh
```

Use a controlled alias-to-canonical lookup. Keep the original service label in an additional source_service column so normalization never destroys evidence. An unknown label remains unchanged and receives review_status=manual_review.

3.3 Boundary tests

Input	Expected result Reason
Port 0 Port 1	Reject Below valid transport port range. Accept if other fields are valid Lowest valid port.
Port 65535 Port 65536	Accept if other fields are valid Highest valid port. Reject Above valid port range.

192.168.56.255	Broadcast address in a /24; Syntactically valid IPv4; scope depends on subnet policy should not be treated as an ordinary host without context.
192.168.56.256	Reject IPv4 octet exceeds 255.

3.4 Before-and-after evidence and safe deletion

```
wc -l ports_testcopy.csv ports_clean.csv rejected.csv
diff -u ports_testcopy.csv ports_clean.csv | head
head -n 10 ports_clean.csv
# Reconcile: input data rows = cleaned rows + rejected rows + duplicate rows logged
```

Every removed or changed row must be traceable. Keep separate files for rejected records, duplicates and manual review. Student A leads missing-value, duplicate and malformed-address rules. Student B leads service-name mapping and verifies that cleaning preserves meaningful findings.

3.5 Why cleaning alone cannot guarantee accuracy

Automated cleaning can enforce formats and known rules, but it cannot establish whether a scan was authorized, whether a host was actually reachable at collection time, whether a service banner is truthful, or whether a version observation is current. A syntactically valid IP can be out of scope; a correctly formatted service name can still be wrong; a duplicate can represent repeated observations at different times. Human review, source provenance, scope checks and controlled re-scanning are therefore necessary.

Q4 – Create a Scan-Result Reshaping Pipeline Using Melting and Casting

4.1 Wide and long representations

In a wide table, each host is a row and each scanned port is a separate column. In a long table, each observation is represented by a row containing host, port and state. The

identifier is host (and scan_id); port is a dimension; state is the measured variable.

Representation	Example Best use
Wide	ip,22,80,443 / 192.168.56.10,open,open,closed	Compact host-by-port display for a fixed port set.
Long	scan_id,ip,protocol,port,state / s01,192.168.56.10,tcp,22,open	Filtering, grouping, joining and analysis across arbitrary ports.

4.2 Example wide-to-long transformation

```
awk -F, 'BEGIN{OFS=","} NR==1{for(i=2;i<=NF;i++) port[i]=$i; next} \
 {for(i=2;i<=NF;i++) print $1,"tcp",port[i],$i}' wide.csv > long.csv
```

This template assumes a simple CSV with no quoted commas and a header containing port numbers. For production input, use a CSV-aware parser. Validate that every host/port cell becomes exactly one long-form record.

4.3 Casting and summary

```
# Count open ports per host from long.csv (columns: ip,protocol,port,state) awk
-F, 'NR>1 && $4=="open" {count[$1]++} END {for (h in count) print h,count[h]}'
long.csv | sort
```

Validation	Method Expected invariant
Row-count preservation	Compare number of non-header wide cells with long rows.	Long row count equals hosts multiplied by measured port columns, excluding deliberately omitted cells.
Round-trip	Cast long data back to the same host/port layout.	Every host/port state matches the original.
Checksum	Sort canonicalized records and compute sha256sum.	Equal canonical content

	produces equal checksum.

Totals Count open states in long data and compare with cast	summary. Sum of per-host open counts	equals total open observations.

4.4 Why more detail may reduce clarity

A full scan of all 65,535 TCP ports per host can produce a very large dataset; UDP is a separate scan space. A long table grows with hosts multiplied by queried ports and can overwhelm a management reader. Detailed raw evidence is valuable for technical investigation, while a summary should expose the decision-relevant facts such as open services, affected hosts and confidence. Keep both levels and link summaries back to source rows.

Student A implements wide-to-long melting and explains identifiers/dimensions/measures. Student B creates the cast summary and reconciles totals. Record actual Nmap elapsed time and wc -l results from the lab; do not infer experimental results from theoretical complexity.

Q5 – Design a Multi-Stage Scan-Data Merging Strategy

5.1 Keys and merge strategy

The master scan table is built by joining normalized datasets using stable keys. Hosts join on scan_id and IP; Ports and Services join on scan_id, IP, protocol and port; OS/Versions and host-level Enumeration join on scan_id and IP. The protocol field is essential because TCP port 53 and UDP port 53 are different observations.

Join type	**Meaning**	**Appropriate use Risk**

Inner join	Retains only matching keys.	Build a table of services with corresponding port records.	Drops unmatched findings unless separately captured.
Left join (-a1)	Retains all records from the first input and matching second-side values.	Keep every host while attaching available port/service data.	Missing child values must be interpreted explicitly.
Full outer join (-a1 -a2)	Retains matched and unmatched records from both inputs.	Reconciliation and exception analysis.	Identify orphan ports, of absent fields. Unmatched does not automatically mean invalid; investigate source and scope.
Unmatched (-v)	Returns records without a match.	Requires clear handling services or host observations.	

5.2 Staged merge sequence

- Validate and sort each dataset by its join key.
- Join Hosts to Ports using scan_id and IP, preserving hosts with no discovered ports where required.
- Join Services to Ports using scan_id, IP, protocol and port.
- Attach OS/Versions and host-level Enumeration using scan_id and IP.
- Retain unmatched outputs from every stage for investigation.
- Compare row counts and unique keys before and after each join; investigate unexpected multiplication.

5.3 Example shell reconciliation

```
sort -t, -k1,1 -k2,2 hosts.csv > hosts.sorted.csv
sort -t, -k1,1 -k2,2 ports.csv > ports.sorted.csv
# For a simple two-column demonstration after preparing matching key files:
join -t, -1 1 -2 1 hosts_key.csv ports_key.csv > matched.csv
join -t, -1 1 -2 1 -v 1 hosts_key.csv ports_key.csv > hosts_without_ports.csv
join -t, -1 1 -2 1 -v 2 hosts_key.csv ports_key.csv > ports_without_hosts.csv
```

GNU join requires both inputs sorted on the join field and compatible field formats. For

composite keys, create a normalized single key column such as scan_id|ip|protocol|port, or use an awk associative array / a CSV-aware data tool. Do not join only on port number.

5.4 Row multiplication and join correctness

Symptom	Possible cause Check
Output rows exceed expected rows	Duplicate join keys create many-to-many matches. Count duplicate keys before joining; inspect sort | uniq -d.
Unmatched service rows	Service observation has no corresponding port key. Use join -v and check protocol, scan_id and port consistency.
Hosts disappear	Inner join used where left preservation was required. Compare distinct host count before and after; use left/full reconciliation.
Unexpectedly empty join	Key formatting or sort order differs. Normalize whitespace, delimiter and key order; verify sort locale.

Student A leads host-service and host-port joins. Student B leads OS/version and enumeration integration. Both must explain why join types produce different counts and provide count-based evidence that the master table is correct.

Q6 – Create an Automated Scan-File Reading and Writing Pipeline

6.1 Import pipeline

- Accept a source path, dataset type, scan_id and approved scope list as explicit

arguments.

- Check that the file exists and is readable; identify XML, grepable Nmap or CSV input.
- Convert structured Nmap XML using xsltproc or a documented parser; preserve original XML.
- Validate required headers and data types before transformation.
- Write normalized data to a new versioned output path; never overwrite raw evidence.
- Re-import the written output and compare row counts and checksums.

```
xsltproc nmap.xsl scan.xml > scan_export.csv
# Example naming pattern:
subnet_scantype_YYYYMMDD-HHMMSS_vN.csv
set -o noclobber
sha256sum scan_export.csv > scan_export.csv.sha256
sha256sum -c scan_export.csv.sha256
cmp scan_export.csv scan_export_readback.csv
```

6.2 Schema and type checks

Check	**Example Failure behavior**
Header	Expected exact ordered column names. Stop import and report missing/unexpected columns.
IP	Parse with a real IP validator or Python ipaddress. Reject malformed address and preserve row.
Port	Numeric integer within 1–65535. Reject invalid value.
File type	Extension and content agree (CSV/XML). Do not trust extension alone; inspect structure.

Scan metadata	scan_id, target scope, timestamp and command present. Quarantine incomplete provenance.

6.3 Versioning and read-back validation

Use a deterministic naming pattern containing subnet, scan type, timestamp and version. Store a manifest with command, tool version, scope, input filenames, output schema version and checksums. Use noclobber or an explicit no-overwrite check. Read-back validation should compare headers, row counts, canonicalized content and SHA-256 checksums.

Evidence	**Command / method What it demonstrates**
File existence	test -r input.csv Input is present and readable.
Line count	wc -l input.csv output.csv Gross row-count reconciliation.
Content equality	cmp file_a file_b / diff -u Byte-level or human-readable differences.
Integrity	sha256sum file; sha256sum -c checksumfile File has not changed since checksum creation.
Schema consistency	Header comparison script Export retains the intended columns and order.

Student A implements import and schema/type checks. Student B implements export, versioning and read-back checks. Both retain raw inputs and demonstrate that rerunning does not overwrite earlier results.

Q7 – Develop a Reusable Scanning and Enumeration Function Library

7.1 Library design principles

The shared Bash library, scanlib.sh, should expose small functions with explicit

arguments, safe defaults, status-code returns and errors written to stderr. The library should not launch scans automatically when sourced. Scanning functions should require an explicit target and should verify that the target is within the supplied authorization list.

```
#!/usr/bin/env bash
# scanlib.sh – reusable functions; sourcing this file performs no scan.
set -o pipefail

log_error() { printf 'ERROR: %s\\n' "$*" >&2; }
require_file() {
 [[ -r "$1" ]] || { log_error "file not readable: $1"; return 2; }
}
validate_port_csv() {
 local input="${1:-ports.csv}"
 require_file "$input" || return $?
 awk -F, 'NR==1{next} $4 !~ /^[0-9]+$/ || $4<1 || $4>65535 {bad=1} END{exit bad}'
"$input" \
 || { log_error "invalid port data in $input"; return 3; }
}
clean_duplicates() {
 local input="${1:-ports.csv}" output="${2:-ports_clean.csv}"
 require_file "$input" || return $?
 [[ ! -e "$output" ]] || { log_error "refusing to overwrite $output"; return 4; }  {
head -n 1 "$input"; tail -n +2 "$input" | sort -t, -k1,1 -k2,2 -k3,3 -k4,4 -u; } >
"$output"
}
export_checksum() {
 local input="${1:-master.csv}"
 require_file "$input" || return $?
 sha256sum "$input"
}
```

This starter library illustrates error handling and safe file behavior. A completed pair implementation should add the import, reshape, merge, export, summary and authorized scan-wrapper functions required by the brief. Use set -euo pipefail in the driver script; in a sourced library, avoid changing the caller's shell options unexpectedly.

7.2 Function contract

Function	Inputs / defaults Success / failure behavior
validate_schema	input_file, schema_name 0 when valid; non-zero and stderr explanation when invalid.
validate_port_csv	input_file default ports.csv Rejects malformed port values and reports failure.

clean_duplicates	input_file, output_file	Writes normalized output without overwriting an existing file.
reshape_wide_to_long	input_file, output_file	Writes host/port/state rows and preserves source identifiers.
merge_scan_data	hosts, ports, services, output	Creates master dataset and unmatched exception files.
write_versioned_output	dataset, directory, scan metadata	Creates unique timestamp/version output and checksum.
summarize_scan	master_file, output_directory	Creates host and service counts reconciled to master data.

7.3 Six-stage collaborative implementation

Stage	**Student A – primary responsibility**	**Student B – primary responsibility**	**Joint evidence**
1. Independent proposal	Validation and cleaning functions; inputs, defaults, return values.	Reshape, merge and file-handling functions; error handling.	Retain both initial proposals.
2. Integration	Import, validation, cleaning; input/rejected/cleaned counts.	Reshape, merge, export; merged/final counts and repeatability.	Both run integrated pipeline.

3. Reporting 4. Stress test 5. Changed requirement	Per-host and per-service summaries; reconcile totals. Prepare 50 and 5,000 host-port record datasets; baseline benchmark. Schema and key redesign for UDP/new hosts.	Exposure/version-risk summary and management interpretation. Identify bottleneck and implement one optimization. Multi-session merge and change validation with ndiff/comm. Master-table reconciliation. Before/after correctness and performance. Compare original and revised pipeline.
6. Joint evaluation	Explain validation, cleaning and schema decisions.	Explain transformations, merging and reporting decisions. Defend reliability, maintainability and trade-offs.

7.4 Stress testing and optimization

Use generated synthetic CSV records for scale tests rather than scanning thousands of live hosts. Prepare at least the two scales named in the brief: 50 and 5,000 host-port records. Run the same workload multiple times and record median elapsed time, peak memory, output row count and checksum. A possible optimization is replacing a per-line grep loop with one awk pass. Compare output correctness before comparing speed.

```
/usr/bin/time -v bash pipeline_baseline.sh input_5000.csv
/usr/bin/time -v bash pipeline_optimized.sh input_5000.csv
sha256sum baseline_output.csv optimized_output.csv
wc -l baseline_output.csv optimized_output.csv
```

Do not report a speedup unless both versions produce equivalent canonical output. Record machine specifications, OS, shell/tool versions and number of repetitions. A

single timing is vulnerable to caching and background load.

7.5 Changed requirement: UDP and multiple scan sessions When UDP services are introduced, the port key must include protocol: scan_id + IP + protocol + port. Without protocol, TCP/53 and UDP/53 could collide. Newly discovered hosts should be inserted as new host observations, not silently overwrite earlier sessions. Keep scan_id and observed_at so changes can be compared across runs.

Session comparison state	Interpretation Pipeline action
Retained host	Present in both old and new authorized sessions. Link observations and compare changed services/versions.
Updated host	Same IP but changed observed attributes. Store new observation; preserve previous session.
Unmatched/removed host	Present previously but absent in the new scan. Mark not observed in latest session; do not assume decommissioned.
New host	Appears in latest authorized scan only. Validate scope and add as a new host observation.

```
ndiff previous_scan.xml current_scan.xml
comm -23 previous_hosts.txt current_hosts.txt # previous-only entries
comm -13 previous_hosts.txt current_hosts.txt # current-only entries
```

7.6 Joint evaluation and final engineering judgment

A structured sequence is necessary because each stage depends on guarantees established by the previous stage. Import establishes a controlled input boundary; validation checks schema and scope; cleaning standardizes known defects; reshaping supports analysis; merging connects observations; export and checksums preserve

traceability; reporting turns technical records into decision-useful summaries.

- Modular functions improve reuse, testing and maintainability, but require stable interfaces and clear error contracts.
- Validation and provenance checks add processing cost but reduce incorrect conclusions and scope violations.
- Repeated scans improve temporal understanding but create larger datasets and require session-aware keys.
- Joins can create row multiplication when keys are not unique; composite keys and reconciliation are essential.
- Pipeline suitability must be reviewed when new protocols, operating systems, scan tools or output formats are introduced.
- A pipeline becomes unreliable if it silently drops rows, guesses missing values, overwrites raw evidence or treats unverified observations as facts.

A reliable scan-data pipeline should be authorized, reproducible, auditable, schema-aware, conservative when correcting data, efficient at scale and adaptable to new scan sources. It should preserve raw evidence and make every transformation explainable.

Contribution Record

Complete this table at the end of the project. It must reflect actual work performed by each student; primary contribution does not mean exclusive responsibility.

Task	**Student A – primary contribution**	**Student B – primary contribution**	**Joint work / evidence**
Q1 – Requirements and data architecture	Defined the Hosts and Ports schemas, required fields and primary-key mapping.	Defined Services, OS/Versions and Enumeration relationships and the	Reviewed the complete schema, keys, relationships and data dictionary together.

		enquiry requirements.
Q2 – Source data and validation	Prepared the Hosts and Ports source-data structure and	Prepared the Services, OS/Versions and Enumeration data structure; developed Checked source provenance, scope, expected validation

	documented controlled test defects.	validation and standardization checks. outcomes and the test dataset.
Q3 – Data cleaning and editing	Led missing-value, duplicate-record and malformed-addres s handling rules.	Led service-name normalization and checked that cleaning preserved meaningful scan findings. Reviewed rejected, corrected and manual-review records and maintained traceability.
Q4 – Melting and casting	Implemented and explained wide-to-long melting, including identifiers, dimensions and measures.	Prepared the cast summary and reconciled transformed totals against the source data. Compared row counts and documented the transformation and reconciliation procedure.
Q5 – Joining and merging Q6 – Automated import and export	Led host-service and host-port joins and checked key uniqueness and unmatched records. Led input import,	Led OS/version and enumeration integration and reviewed join outcomes. Compared join types, checked row multiplication and reconciled the master

	schema checks, data-type validation and input/rejection checks.	table. Led export, file versioning, naming conventions and read-back validation. Reviewed repeatability, preservation of raw inputs and end-to-end pipeline consistency.
Q7 – Reusable function library and integration	Led validation and cleaning functions, including inputs, defaults, return values and error checks.	Led reshape, merge and file-handling functions, including error handling and optimization. Integrated and tested the function library, reviewed stress-test evidence and jointly explained the changed-requirement updates.

Final Submission Checklist

● Both students' initial Q7 function proposals are retained.

● Authorized target list, lab topology and scope boundaries are documented. ● Raw Nmap XML/grepable outputs and original CSV files are preserved. ● Defect-injected copies are clearly separated from original evidence. ● Before-and-after cleaning, rejected and manual-review files are included. ● Melting/casting row counts, round-trip comparison and checksums are included. ● Join alternatives, unmatched records and row-count reconciliation are demonstrated. ● Import/export naming, versioning and read-back validation are demonstrated.

● Q7 function library, integration tests, stress tests and changed-requirement tests are included.

● Actual performance measurements and command outputs are attached; no results are fabricated.

References

● Nmap Project. Nmap Reference Guide: https://nmap.org/book/man.html ● Nmap Project. Output Formats: https://nmap.org/book/output.html ● GNU Bash Reference Manual: https://www.gnu.org/software/bash/manual/ ● GNU Coreutils Manual: https://www.gnu.org/software/coreutils/manual/ ● GNU Awk User's Guide: https://www.gnu.org/software/gawk/manual/ ● GNU Findutils / Diffutils documentation: https://www.gnu.org/software/diffutils/manual/ ● xsltproc documentation: https://gnome.pages.gitlab.gnome.org/libxslt/xsltproc.html ● OpenSSH / SHA-2 checksum utilities documentation and local manual pages (sha256sum, sort, join, comm, diff, cmp, wc).